

```

#include <IRremote.hpp>
#include <NewPing.h>
#include <Servo.h>

//===== IR =====

#define IR_PIN A0

// Thay bằng mã remote của bạn
#define IR_FORWARD 0xBA45FF00
#define IR_BACKWARD 0xB946FF00
#define IR_LEFT 0xBB44FF00
#define IR_RIGHT 0xBC43FF00
#define IR_STOP 0xBF40FF00

#define IR_MODE0 0xF30CFF00
#define IR_MODE1 0xE718FF00
#define IR_MODE2 0xA15EFF00

//===== MOTOR =====

const int enAPin = 6;
const int in1Pin = 7;
const int in2Pin = 5;
const int in3Pin = 4;
const int in4Pin = 2;
const int enBPin = 3;

int speedCar = 180;

//===== LINE SENSOR =====

```

```

const int SensorLeft  = 10;
const int SensorMiddle = 9;
const int SensorRight = 8;

//===== ULTRASONIC =====

#define TRIG_PIN 13
#define ECHO_PIN 12
#define MAX_DISTANCE 200

Servo myservo;
NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

//===== MODE =====

#define MODE_MANUAL  0
#define MODE_LINE    1
#define MODE_OBSTACLE 2

int MODE = MODE_MANUAL;

//=====

void setup()
{
  pinMode(enAPin, OUTPUT);
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
  pinMode(enBPin, OUTPUT);

```

```
pinMode(SensorLeft, INPUT);  
pinMode(SensorMiddle, INPUT);  
pinMode(SensorRight, INPUT);
```

```
Serial.begin(9600);
```

```
IrReceiver.begin(IR_PIN, ENABLE_LED_FEEDBACK);
```

```
myservo.attach(11);  
myservo.write(95);
```

```
stopRobot();
```

```
Serial.println("Robot Ready");
```

```
}
```

```
//=====
```

```
void loop()
```

```
{
```

```
  readIR();
```

```
  readBluetooth();
```

```
  switch (MODE)
```

```
  {
```

```
    case MODE_LINE:
```

```
      lineFollower();
```

```
      break;
```

```

    case MODE_OBSTACLE:
        obstacleAvoid();
        break;
    }
}

//=====

// IR CONTROL

//=====

void readIR()
{
    if (IrReceiver.decode())
    {
        uint32_t code =
            IrReceiver.decodedIRData.decodedRawData;

        Serial.println(code, HEX);

        switch (code)
        {
            case IR_MODE0:
                MODE = MODE_MANUAL;
                stopRobot();
                break;

            case IR_MODE1:
                MODE = MODE_LINE;

```

```
break;
```

```
case IR_MODE2:
```

```
    MODE = MODE_OBSTACLE;
```

```
    break;
```

```
case IR_FORWARD:
```

```
    if (MODE == MODE_MANUAL)
```

```
        goAhead();
```

```
    break;
```

```
case IR_BACKWARD:
```

```
    if (MODE == MODE_MANUAL)
```

```
        goBack();
```

```
    break;
```

```
case IR_LEFT:
```

```
    if (MODE == MODE_MANUAL)
```

```
        goLeft();
```

```
    break;
```

```
case IR_RIGHT:
```

```
    if (MODE == MODE_MANUAL)
```

```
        goRight();
```

```
    break;
```

```
case IR_STOP:
```

```
    stopRobot();
```

```
    break;
```

```

    }

    IrReceiver.resume();
}
}

//=====
// BLUETOOTH CONTROL
//=====

void readBluetooth()
{
    if (Serial.available())
    {
        char cmd = Serial.read();

        switch (cmd)
        {
            case 'M':
                MODE = MODE_MANUAL;
                stopRobot();
                break;

            case 'T':
                MODE = MODE_LINE;
                break;

            case 'A':
                MODE = MODE_OBSTACLE;

```

```
break;
```

```
case 'F':
```

```
if (MODE == MODE_MANUAL)
```

```
    goAhead();
```

```
break;
```

```
case 'B':
```

```
if (MODE == MODE_MANUAL)
```

```
    goBack();
```

```
break;
```

```
case 'L':
```

```
if (MODE == MODE_MANUAL)
```

```
    goLeft();
```

```
break;
```

```
case 'R':
```

```
if (MODE == MODE_MANUAL)
```

```
    goRight();
```

```
break;
```

```
case 'S':
```

```
    stopRobot();
```

```
break;
```

```
case 'O':
```

```
    speedCar = 100;
```

```
break;
```

```
case '1':  
    speedCar = 120;  
    break;
```

```
case '2':  
    speedCar = 140;  
    break;
```

```
case '3':  
    speedCar = 160;  
    break;
```

```
case '4':  
    speedCar = 180;  
    break;
```

```
case '5':  
    speedCar = 200;  
    break;
```

```
case '6':  
    speedCar = 220;  
    break;
```

```
case '7':  
    speedCar = 255;  
    break;
```

```
}
```



```

    }
}

//=====

// LINE FOLLOWER

//=====

void lineFollower()
{
    int SL = digitalRead(SensorLeft);
    int SM = digitalRead(SensorMiddle);
    int SR = digitalRead(SensorRight);

    if (SM && SR && !SL)
    {
        goAhead(160, 40);
    }
    else if (SM && !SR && SL)
    {
        goAhead(40, 160);
    }
    else if (SM && !SR && !SL)
    {
        goAhead(160, 160);
    }
    else if (!SM && !SR && SL)
    {
        goAhead(80, 200);
    }
}

```

```
else if (!SM && SR && !SL)
{
    goAhead(200, 80);
}
else
{
    stopRobot();
}
}
```

```
//=====
// OBSTACLE AVOID
//=====
```

```
void obstacleAvoid()
{
    int distance = readPing();

    if (distance <= 25)
    {
        stopRobot();
        delay(100);

        goBack();
        delay(250);

        stopRobot();
        delay(100);
    }
}
```

```
int rightDistance = lookRight();  
delay(100);
```

```
int leftDistance = lookLeft();  
delay(100);
```

```
if (rightDistance > leftDistance)  
{  
    goRight();  
    delay(300);  
}  
else  
{  
    goLeft();  
    delay(300);  
}
```

```
    stopRobot();  
}  
else  
{  
    goAhead();  
}  
}
```

```
//=====
```

```
// ULTRASONIC
```

```
//=====
```

```
int readPing()
{
    delay(50);

    int cm = sonar.ping_cm();

    if (cm == 0)
        cm = 250;

    return cm;
}
```

```
int lookRight()
{
    myservo.write(40);
    delay(500);

    int d = readPing();

    myservo.write(95);

    return d;
}
```

```
int lookLeft()
{
    myservo.write(150);
    delay(500);
```

```

int d = readPing();

myservo.write(95);

return d;
}

//=====
// MOTOR
//=====

void goAhead()
{
    digitalWrite(in1Pin, LOW);
    digitalWrite(in2Pin, HIGH);
    digitalWrite(in3Pin, LOW);
    digitalWrite(in4Pin, HIGH);

    analogWrite(enAPin, speedCar);
    analogWrite(enBPin, speedCar);
}

void goBack()
{
    digitalWrite(in1Pin, HIGH);
    digitalWrite(in2Pin, LOW);
    digitalWrite(in3Pin, HIGH);
    digitalWrite(in4Pin, LOW);
}

```

```
    analogWrite(enAPin, speedCar);  
    analogWrite(enBPin, speedCar);  
}
```

```
void goLeft()  
{  
    digitalWrite(in1Pin, HIGH);  
    digitalWrite(in2Pin, LOW);  
    digitalWrite(in3Pin, LOW);  
    digitalWrite(in4Pin, HIGH);  
  
    analogWrite(enAPin, speedCar);  
    analogWrite(enBPin, speedCar);  
}
```

```
void goRight()  
{  
    digitalWrite(in1Pin, LOW);  
    digitalWrite(in2Pin, HIGH);  
    digitalWrite(in3Pin, HIGH);  
    digitalWrite(in4Pin, LOW);  
  
    analogWrite(enAPin, speedCar);  
    analogWrite(enBPin, speedCar);  
}
```

```
void goAhead(int leftSpeed, int rightSpeed)  
{  
    digitalWrite(in1Pin, LOW);
```

```
digitalWrite(in2Pin, HIGH);  
digitalWrite(in3Pin, LOW);  
digitalWrite(in4Pin, HIGH);  
  
analogWrite(enAPin, leftSpeed);  
analogWrite(enBPin, rightSpeed);  
}
```

```
void stopRobot()  
{  
    digitalWrite(in1Pin, LOW);  
    digitalWrite(in2Pin, LOW);  
    digitalWrite(in3Pin, LOW);  
    digitalWrite(in4Pin, LOW);  
  
    analogWrite(enAPin, 0);  
    analogWrite(enBPin, 0);  
}
```